

Editor Extensions Verification Checklist

Verification checklist for Unity Editor extensions added or updated between commit

`33b2c6053118d8e33dd785ee797693348d76407c` and `HEAD`.

Note: Use this checklist together with the [Sample Scene Verification Checklist](#). Verifying Play-only tools requires Play-capable scenes such as `MidiSampleScene` or `MidiGameplaySampleScene`.

Common prerequisites

- ☐ Open the project in the Unity Editor with **no compile errors in the Console**
- ☐ Each item under the **Window > MIDI** menu is visible
- ☐ Before verifying Play-only tools, prepare a sample scene that calls `MidiManager.InitializeMidi()`
- ☐ Optionally use **Window > MIDI > Monitor** to confirm IN/OUT

Scripting Define Symbols

Symbol	Target editor extension
<code>FEATURE_USE_MPTK</code> + Maestro / MPTK asset	Preview Audio (MPTK) in SMF Preview
<code>FEATURE_USE_TIMELINE</code> + <code>com.unity.timeline</code>	Timeline integration editors (marker generation, Clip / Marker Inspectors, etc.)
<code>FEATURE_USE_VISUALSCRIPTING</code> + <code>com.unity.visualscripting</code>	Visual Scripting menu, node registration

Added / updated Editor extensions

Core editor tools ([Window](#) > [MIDI](#))

#	Tool	Menu	Mode	Main sources
1	MIDI Monitor (updated)	Window > MIDI > Monitor	Play only	MidiMonitorWindow.cs and others
2	Virtual MIDI Controller (new)	Window > MIDI > Virtual Controller	Play only	VirtualMidiControllerWindow.cs
3	SMF Preview (new)	Window > MIDI > SMF Preview	Edit only	SmfPreviewWindow.cs
4	SMF Import (new)	Assets > Import MIDI File...	Edit only	SmfImportUtility.cs
5	Project Settings (new)	Edit > Project Settings > MIDI	Edit	MidiProjectSettingsProvider.cs

Unity integration editors

#	Tool	Menu	Requires
6	Generate integration sample assets	Window > MIDI > Samples > Generate Integration Sample Assets	None
7	Animator Driver Inspector	Inspector (MidiAnimatorDriver)	None
8	Timeline marker generation	Window > MIDI > Timeline > Generate Markers From Sequence Asset	FEATURE_USE_TIMELINE
9	Add Timeline Notification Receiver	Window > MIDI > Timeline > Add Notification Receiver To Director	FEATURE_USE_TIMELINE
10	Visual Scripting helpers	Window > MIDI > Visual Scripting/*	FEATURE_USE_VISUALSCRIPTING

Recommended test order

1. **Project Settings** — auto-create on first import, settings UI
2. **SMF Preview / Import** — Edit-mode-only features
3. **MIDI Monitor** — Play-mode IN/OUT capture, filters, export
4. **Virtual Controller** — send verification alongside Monitor
5. **SMF Preview Audio (MPTK)** — Play-mode audition when MPTK is enabled
6. **Unity integration editors** — Animator / Timeline / Visual Scripting

Per-tool checklist

1. Project Settings ([Edit](#) > [Project Settings](#) > [MIDI](#))

Purpose: Manage global MIDI settings; auto-create the settings asset on first import

- ☐ Opening the project auto-creates [Assets/MIDI/Resources/MidiProjectSettings.asset](#)
 - ☐ **Devices** section: Default Input / Output Device Id can be edited and saved
 - ☐ **Bluetooth MIDI**: Auto Scan / Timeout can be changed
 - ☐ **RTP-MIDI**: Port / Session Name can be changed
 - ☐ **Debug**: Log Level / Enable MIDI Monitor On Play can be changed
 - ☐ **Development**: Development Build Only Verbose Log can be changed
 - ☐ During Play mode, Default Device can be chosen from connected devices via dropdown
 - ☐ When [enableMidiMonitorOnPlay](#) = [true](#), Monitor opens automatically on Play
-

2. SMF Preview ([Window](#) > [MIDI](#) > [SMF Preview](#))

Purpose: Preview and export [.mid](#) files in Edit mode

File loading

- ☐ Opening the window shows the drop area "Drop .mid file here"
- ☐ Drag and drop [.mid](#) / [.midi](#) → summary (Format / Ticks/Quarter / Tracks / Length) appears
- ☐ Choosing a file via [Assets > Import MIDI File...](#) opens SMF Preview and loads it
- ☐ Loading an invalid file shows an error message in a HelpBox

Content display

- ☐ Selecting a track in **Tracks** switches the **Events** list
- ☐ Events columns (Tick / Time / Type / Ch / Detail) display correctly
- ☐ **Note Roll (text)** foldout → note list appears

Export

- ☐ **Export as MidiSequenceAsset** → saves [.asset](#) in the Project and Pings it
- ☐ Exported [MidiSequenceAsset](#) can be referenced by [SmfPlayer](#) or a Timeline Clip
- ☐ **Export as JSON** → saves external [.json](#) and shows a status message

Audio preview (MPTK)

Requires: [FEATURE_USE_MPTK](#), Maestro / MPTK asset

- ☐ **Preview Audio (MPTK)** → enters Play mode and starts SMF playback
- ☐ Console shows [\[SMF Preview Playback\] Playing ...](#)
- ☐ **Stop Preview** → playback stops and [SmfPreviewPlaybackHost](#) is destroyed
- ☐ After playback completes, the host GameObject is destroyed automatically

When MPTK is disabled:

- ☐ Audio Preview section shows an Info HelpBox (MPTK required) and no button
-

3. MIDI Monitor (**Window > MIDI > Monitor**)

Purpose: Real-time MIDI IN/OUT display and log export during Play

Requires: Play mode (messages are not recorded in Edit mode)

Basic display

- ☐ Play **MidiSampleScene** (or similar) and open Monitor
- ☐ IN: Note On/Off / CC / PC appear from hardware input or Inject
- ☐ OUT: messages from **MidiSend** or Virtual Controller appear
- ☐ Time / Dir / Device / Ch / Type / Detail columns appear
- ☐ Note Detail includes note names (e.g. **C4 (60)**)
- ☐ Device connect / disconnect events appear as Device type

Toolbar

- ☐ **Clear** → log is cleared
- ☐ **Auto Scroll** → scrolls to the end when new rows are added
- ☐ Changing **Max Lines** limits retained rows (1–10000)

Filters

- ☐ **Direction:** All / IN / OUT
- ☐ **Device:** filter by connected device
- ☐ **Ch:** channel filter
- ☐ **Type:** NoteOn / NoteOff / CC / PC / Device
- ☐ **Search:** partial match on Detail / Device
- ☐ Closing and reopening the window restores filter settings from EditorPrefs

Column resize

- ☐ Dragging the right edge of a column header changes column width
- ☐ Closing and reopening the window preserves column widths

Export

- ☐ **Export CSV** → saves filtered log as **.csv**
- ☐ **Export TXT** → saves filtered log as **.txt**
- ☐ **Export SMF** → saves filtered OUT events as **.mid**
- ☐ When export is not possible (no events), an error message appears in status
- ☐ Exported SMF can be reloaded in SMF Preview

Lifecycle

- ☐ After Play ends, log content remains in the window

- ☐ Playing again appends new messages (clearable with Clear)
-

4. Virtual MIDI Controller ([Window > MIDI > Virtual Controller](#))

Purpose: Send Note / CC / PC / Pitch Bend without a physical device

Requires: Play mode, [MidiManager.InitializeMidi\(\)](#) already called

Edit mode

- ☐ In Edit mode, a HelpBox "Play mode only" is shown and controls are disabled

Play mode — devices

- ☐ Output device list appears (Project Settings default / connected devices / [editor:virtual-controller](#))
- ☐ When nothing is connected, the [editor:virtual-controller](#) virtual output is registered automatically
- ☐ Channel / Group / Velocity settings are applied

Play mode — keyboard

- ☐ Clicking the 2-octave keyboard (C3–B4) → Note On
- ☐ Releasing the mouse → Note Off
- ☐ Monitor OUT shows Note On/Off

Play mode — CC / other

- ☐ Operating the 16 CC sliders → CC send (confirm in Monitor OUT)
- ☐ CC number settings are saved/restored via EditorPrefs
- ☐ Program Change / Pitch Bend send (UI section)
- ☐ **All Notes Off** → CC 123 on all channels
- ☐ **Panic** → CC 120 + 123 on all channels

End of Play

- ☐ When Play ends, held notes are All Off'd
-

5. Generate integration sample assets

Menu: [Window > MIDI > Samples > Generate Integration Sample Assets](#)

- ☐ Running the menu → completion dialog appears
 - ☐ [Assets/MIDI/Samples/Integrations/Resources/MidiAnimatorSample.controller](#) is generated
 - ☐ Controller includes Height / Pulse / BlendX / BlendY parameters
 - ☐ Re-running overwrites and regenerates the existing Controller
-

6. MidiAnimatorDriver Inspector

Purpose: Edit Animator bindings; show live values during Play

Steps:

1. Open `MidiAnimatorIntegrationSampleScene`, or select a GameObject with `MidiAnimatorDriver`
 2. Inspect the Inspector
 - ☐ Animator / Mapping / CC Smoother / Register With MidiManager fields appear
 - ☐ When Mapping is unset, the Bindings array is editable
 - ☐ **Add Binding** / **Remove Last** add/remove bindings
 - ☐ When Mapping is set, a HelpBox states Bindings come from the Asset
 - ☐ When Animator is unset, a HelpBox warning appears
 - ☐ During Play, live values of bound Animator parameters are shown
-

7. Timeline integration editors

Requires: `FEATURE_USE_TIMELINE`, `com.unity.timeline`

MidiPlaybackClip Inspector

- ☐ Selecting a Midi Playback Clip on Timeline → edit Sequence Asset / Tempo BPM / Output Channel / Mute / Solo
- ☐ When Sequence Asset is set, a Tracks-count HelpBox appears

MidiPlaybackClip Timeline display

- ☐ Clip display name follows the Sequence Asset name
- ☐ Mute / Solo show `(Muted)` / `(Solo)` suffixes

Marker Inspector / Timeline display

- ☐ `MidiBarMarker` — Bar Number / time signature, Timeline tooltip
- ☐ `MidiTempoMarker` — BPM, Timeline tooltip
- ☐ `MidiMarker` / `MidiSignalEmitter` — edit outbound message, tooltip

Generate Markers From Sequence Asset

Steps:

1. Select a `MidiSequenceAsset` in the Project
2. Open a PlayableDirector in the Timeline window
3. Run `Window > MIDI > Timeline > Generate Markers From Sequence Asset`
 - ☐ Bar markers (`MidiBarMarker`) are generated
 - ☐ Tempo-change markers (`MidiTempoMarker`) are generated
 - ☐ Undo restores the previous state
 - ☐ When Asset is unselected / Director is unassigned, a dialog guides the user

Add Notification Receiver To Director

- ☐ With a Director selected in Timeline, run the menu
 - ☐ `MidiTimelineNotificationReceiver` component is added
 - ☐ Running again does not duplicate the component
-

8. Visual Scripting helpers

Requires: `FEATURE_USE_VISUALSCRIPTING`

Menu: `Window > MIDI > Visual Scripting/*`

Auto-registration

- ☐ After the project loads, `jp.kshoji.midi.visualscripting` is added automatically to the Visual Scripting Node Library

Open Visual Scripting Settings

- ☐ Opens `Project Settings > Visual Scripting`

Regenerate MIDI Nodes

- ☐ Running → Console shows `[MIDI Visual Scripting] Node library regeneration requested.` (on success)
- ☐ Script Graph shows the **Events > MIDI** node category

Show Node Categories

- ☐ Dialog lists MIDI/Events, MIDI/Send, MIDI/Playback, MIDI/Utility, MIDI/Note

Node behavior (combined check)

- ☐ Play `MidiVisualScriptingIntegrationSampleScene`
 - ☐ Note On / CC events run via Script Graph
-

Cross-tool integration checks

Integration tests that verify cooperation across multiple editor extensions.

A. Virtual Controller → Monitor

- ☐ During Play, sending a Note from Virtual Controller → appears in Monitor OUT

B. SMF Preview → MidiSequenceAsset → Timeline

- ☐ Load `.mid` in SMF Preview → export `MidiSequenceAsset`
- ☐ Assign to a Timeline Midi Playback Clip → synced playback in Play (`MidiTimelineIntegrationSampleScene`)

C. SMF Preview → MPTK audition

- ☐ In SMF Preview, **Preview Audio (MPTK)** → Play mode starts automatically → audio output
- ☐ Confirm OUT messages in Monitor (MPTK virtual output)

D. Monitor → SMF re-import

- ☐ MIDI activity during Play → recorded in Monitor → **Export SMF**
- ☐ Load the exported **.mid** in SMF Preview → event contents match

E. Project Settings → Virtual Controller

- ☐ Change Default Output Device Id → reflected in Virtual Controller device list

Added source files (reference)

File	Role
MidiMonitorWindow.cs	MIDI Monitor UI (CSV/TXT/SMF export, column resize)
MidiMonitorLifecycle.cs	Play-start hook, auto-open Monitor
MidiMonitorLogBuffer.cs	Log buffer
MidiMonitorSettings.cs	Filter / column-width EditorPrefs
MidiMonitorSmfExportUtility.cs	Monitor log → SMF conversion
VirtualMidiControllerWindow.cs	Virtual Controller UI
VirtualMidiControllerState.cs	CC numbers / device ID EditorPrefs
UI/PianoKeyboardElement.cs	Editor piano keyboard UI
UI/CcSliderBankElement.cs	CC slider bank UI
SmfPreviewWindow.cs	SMF Preview window
SmfPreviewModel.cs	Preview data model
SmfImportUtility.cs	SMF load / export
SmfPreviewAudioPlayback.cs	MPTK Play-mode audition
MidiProjectSettingsProvider.cs	Project Settings UI
MidiProjectSettingsBootstrap.cs	First-time Settings asset creation
Integrations/MidiIntegrationSampleSetup.cs	Integration sample Controller generation
Integrations/Animator/MidiAnimatorDriverEditor.cs	Animator Driver Inspector
Integrations/Timeline/*	Timeline Clip / Marker editors, marker generation
Integrations/VisualScripting/MidiVisualScriptingMenu.cs	VS node registration menu

Related documents

- [Editor Tools](#)
- [Sample Scene Verification Checklist](#)
- [Unity Ecosystem Integrations](#)
- [SMF Tools](#)
- [Maestro / MPTK Integration](#)